

CS281

Virtual Development Board

Technical Reference Manual
RISC-V RV32IM Educational Platform

Version 1.0 — Drexel University — CS281 Computer Architecture

Table of Contents

1. Introduction

The CS281 Virtual Development Board is a RISC-V RV32IM microcontroller platform emulated in Renode. It provides a realistic, bare-metal programming environment without requiring physical hardware. The board is purpose-built for the CS281 Computer Architecture course at Drexel University.

Students write programs in RISC-V assembly (and optionally C) that run directly on the emulated processor. Every peripheral is accessed through memory-mapped registers at fixed addresses, exactly as on a physical chip. The Renode simulator executes unmodified ELF binaries and exposes GDB-based debugging so students can pause execution, inspect registers, and step through code.

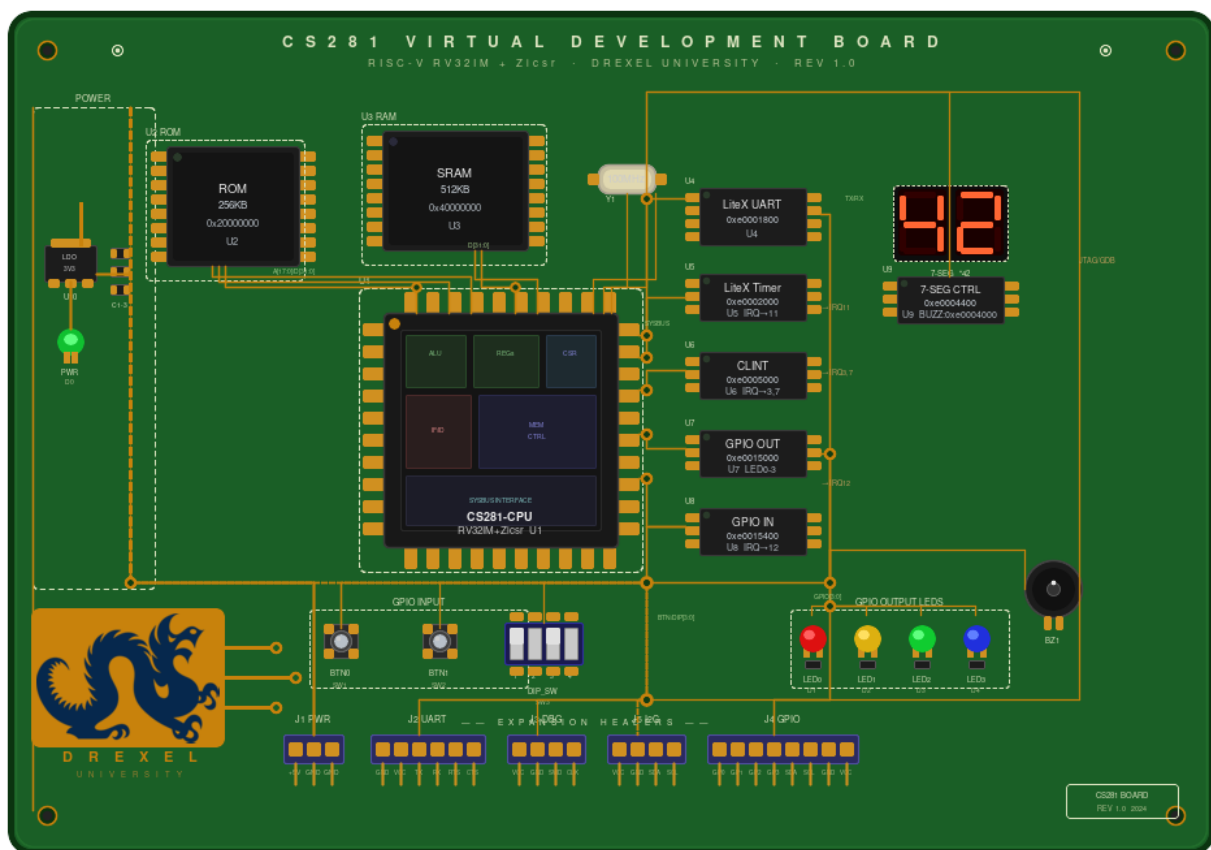


Figure 1 — CS281 Virtual Development Board (PCB rendering)

1.1 Processor Overview

Parameter	Value
Architecture	RISC-V (RV32IM + Zicsr)
Data width	32-bit
Extensions	M — integer multiply/divide; Zicsr — CSR instructions
Privilege level	Machine mode (M-mode) only
Byte order	Little-endian

Parameter	Value
Reset address	Set by ELF entry point (<code>_start</code> at 0x20000000)
Interrupt model	CLINT (software + timer) + external IRQs via <code>cpu @ n</code>

NOTE RV32I instructions always take priority in decoding. The M extension adds `mul`, `mulh`, `div`, `divu`, `rem`, `remu`. The Zicsr extension adds `csrr`, `csrw`, `csrrs`, etc. Compressed (C) and Atomic (A) extensions are NOT present; do not use them.

2. Memory Map

All addresses are physical addresses on the system bus. The processor accesses every peripheral through normal load/store instructions (`lw`, `sw`, `lb`, `sb`, etc.) to the addresses listed below.

Base Address	Name	Size	Access	Description
0x20000000	ROM (Flash)	256 KB	R / X	Program code and read-only data. Loaded from ELF at startup. Treated as read-only by the linker.
0x40000000	RAM	512 KB	R / W / X	Initialized data (<code>.data</code>), zero-initialized data (<code>.bss</code>), heap, and stack.
0xe0001800	UART	32 bytes	R / W	LiteX serial UART — main console I/O.
0xe0002000	Timer	32 bytes	R / W	LiteX countdown timer with interrupt output.
0xe0004000	Buzzer GPIO	4 bytes	W	Single-bit GPIO output driving the board buzzer.
0xe0004400	7-Segment GPIO	4 bytes	W	8-bit GPIO output driving the 7-segment display (segments a–g + dp).
0xe0005000	CLINT	64 KB	R / W	Core-Level Interruptor: machine software (<code>msip</code>) and machine timer (<code>mtime</code> / <code>mtimecmp</code>).
0xe0015000	GPIO Output	4 bytes	W	Four LED outputs (bits 0–3 = LED0–LED3).
0xe0015400	GPIO Input	4 bytes	R	Four board inputs: BTN0, BTN1, DIP_SW0, DIP_SW1 (bits 0–3).

2.1 Memory Sections

The linker script (`hardware/lib/cs281.ld`) maps program sections to the memory regions above. Understanding this layout is essential for writing correct linker scripts and startup code.

ELF Section	Region	Description
<code>.text.startup</code>	ROM (first)	<code>startup.S</code> — must be at the very start of ROM so <code>_start</code> lands at 0x20000000.
<code>.text</code>	ROM	All other compiled/assembled code.

ELF Section	Region	Description
.rodata	ROM	String literals and other read-only constants.
.data	ROM → RAM	Initialized global/static variables. Stored (LMA) in ROM after .rodata; copied to RAM (VMA) by startup.S at boot.
.bss	RAM	Zero-initialized globals and statics. Not stored in ROM; startup.S writes zeros to this region.
(stack)	RAM (top)	Stack grows downward from <code>_stack_top = 0x40080000</code> (top of RAM). No explicit section — the linker exports the symbol only.

3. Boot Sequence and Programmer's Model

When a program is loaded into Renode (via sysbus LoadELF), the simulator sets the CPU program counter to the ELF entry point. The linker script ensures this is `_start`, the first instruction of `startup.S`, located at `0x20000000`.

3.1 Startup Sequence (startup.S)

The startup file in `hardware/lib/startup.S` performs four steps before calling `main()`:

Step	Action	Details
1	Set stack pointer	la sp, <code>_stack_top</code> loads <code>0x40080000</code> into sp. The stack grows downward into RAM.
2	Copy .data to RAM	Reads from ROM (LMA = <code>_etext</code>) and writes to RAM (VMA = <code>_sdata ... _edata</code>). Required because .data has initial values that must be in RAM at runtime.
3	Zero .bss	Writes zero words from <code>_sbss</code> to <code>_ebss</code> . The C standard requires zero-initialised globals; the ELF does not store zeros, so startup must do it.
4	Call main()	call main. If main() returns (it should not), startup spins in an infinite loop (j .Lhalt).

3.2 Linker Symbols

These symbols are defined by `cs281.ld` and used by `startup.S`. They are available in any assembly or C file.

Symbol	Value	Meaning
<code>_etext</code>	(end of .rodata in ROM)	LMA (load address) of the .data section — where .data is stored in ROM.
<code>_sdata</code>	<code>0x40000000 + offset</code>	VMA (run address) start of .data in RAM.
<code>_edata</code>	<code>0x40000000 + offset</code>	VMA end of .data in RAM.
<code>_sbss</code>	(after .data in RAM)	Start of .bss region.
<code>_ebss</code>	(after .bss in RAM)	End of .bss region.

Symbol	Value	Meaning
<code>_stack_top</code>	0x40080000	Top of RAM — initial value of the stack pointer.

NOTE `_etext` marks the boundary between code+rodata and the stored .data image in ROM. It is NOT simply the end of .text — it is the end of all ROM content.

4. Interrupt Reference

The CS281 board uses RISC-V machine-mode (M-mode) interrupts. The CPU's `mstatus.MIE` bit globally enables interrupts; `mie` controls individual sources. When an interrupt fires, the CPU jumps to the address in `mtvec` and the `mcause` register identifies the source.

CPU Line	mcause	Source	Type	Clear Procedure
3	0x80000003	CLINT MSIP	Machine Software	Write 0 to <code>CLINT_BASE + 0x0000</code> (MSIP register).
7	0x80000007	CLINT MTIP	Machine Timer	Write a new value to <code>MTIMECMP</code> greater than the current <code>MTIME</code> .
11	0x8000000B	LiteX Timer	Machine External	Write 1 to <code>Timer EV_PENDING</code> (offset 0x3C) to acknowledge.
12	0x8000000C	GPIO Input	Machine External	Read <code>GPIO_IN</code> to acknowledge. Change must be observed; no separate pending register.

4.1 Enabling Interrupts (Assembly)

Minimal sequence to enable machine-mode interrupts in assembly:

```
# Write the trap vector address to mtvec
la  t0, trap_handler
csrw mtvec, t0

# Enable individual interrupt sources in mie
# Bit 3 = MSIE (software), bit 7 = MTIE (timer), bit 11 = MEIE (external)
li  t0, (1 << 11) | (1 << 7) | (1 << 3)
csrw mie, t0

# Set mstatus.MIE (bit 3) to globally enable interrupts
li  t0, (1 << 3)
csrs mstatus, t0
```

4.2 Trap Handler Template

```
trap_handler:
```

```
# Save caller-saved registers to the stack
addi sp, sp, -64
sw    ra, 0(sp)
sw    t0, 4(sp)
# ... save t1-t6, a0-a7 as needed ...

# Read mcause to identify the interrupt
csrr t0, mcause
# bit 31 = 1 for interrupts, 0 for exceptions
# bits 30:0 = interrupt/exception code (see table above)

# ... dispatch to handler ...

# Restore and return
lw    ra, 0(sp)
lw    t0, 4(sp)
addi sp, sp, 64
mret          # return to interrupted instruction
```

5. UART — Serial Console

The UART peripheral provides a bidirectional serial character interface. It is the primary means of text I/O. In Renode it is connected to a telnet socket on port 3456 (run `make uart-connect` in a second terminal to view output).

Base Address: 0xe0001800

Offset	Register	Access	Reset	Description
0x00	RXTX	R / W	—	Write: transmit one byte (character). Read: receive one byte.
0x04	TXFULL	R	0	1 = TX FIFO is full; do not write to RXTX until 0.
0x08	RXEMPTY	R	1	1 = RX FIFO is empty; no data ready to read.
0x0C	EV_STATUS	R	0	Raw interrupt status flags (TX ready, RX available).
0x10	EV_PENDING	R / W	0	Interrupt pending flags. Write 1 to a bit to clear it.
0x14	EV_ENABLE	R / W	0	Interrupt enable mask. Set bit to route event to the CPU.

5.1 Transmit a Character (Polling)

```

li    t0, 0xe0001800    # UART base
.wait_tx:
    lw    t1, 4(t0)      # read TXFULL
    bnez  t1, .wait_tx   # spin until space available
    sw    a0, 0(t0)      # write character (in a0) to RXTX

```

5.2 Receive a Character (Polling)

```

li    t0, 0xe0001800    # UART base
.wait_rx:
    lw    t1, 8(t0)      # read RXEMPTY
    bnez  t1, .wait_rx   # spin until byte arrives
    lw    a0, 0(t0)      # read character into a0

```

NOTE The library function `uart_putchar(a0)` and `uart_puts(a0)` in `hardware/lib/uart.S` implement polling transmit. Link `uart.S` into your project and call them directly.

6. Timer

The LiteX_Timer is a 32-bit countdown timer running at 100 MHz. It counts down to zero and fires an interrupt on CPU line 11. Important: the peripheral uses an 8-bit CSR data width. The 32-bit LOAD and RELOAD values are each spread across four consecutive 8-bit sub-registers at 4-byte-aligned addresses, written most-significant byte first (sub-register 0 holds bits[31:24], sub-register 3 holds bits[7:0]). Single-bit control registers (EN, EV_PENDING, EV_ENABLE) each occupy one word.

Base Address: 0xe0002000 | Frequency: 100,000,000 Hz | IRQ: CPU line 11

Offset	Register	Width	Description
0x00	LOAD0	8-bit W	Countdown start value bits[31:24] (MSB)
0x04	LOAD1	8-bit W	Countdown start value bits[23:16]
0x08	LOAD2	8-bit W	Countdown start value bits[15:8]
0x0C	LOAD3	8-bit W	Countdown start value bits[7:0] (LSB). Write all four LOAD sub-registers before enabling the timer.
0x10	RELOAD0	8-bit W	Auto-reload value bits[31:24] (MSB). If RELOAD is non-zero the timer reloads automatically; write 0 for one-shot mode.
0x14	RELOAD1	8-bit W	Auto-reload value bits[23:16]
0x18	RELOAD2	8-bit W	Auto-reload value bits[15:8]
0x1C	RELOAD3	8-bit W	Auto-reload value bits[7:0] (LSB)
0x20	EN	8-bit W	Timer enable. Write 1 to start counting, 0 to stop.
0x3C	EV_PENDING	8-bit RW	Interrupt pending flag. Write 1 in your ISR to clear (acknowledge) the interrupt. Failure to clear causes the CPU to re-enter the ISR immediately after mret.
0x40	EV_ENABLE	8-bit W	Write 1 to route the timer interrupt to CPU line 11. Write 0 to mask it.

6.1 Periodic Interrupt Timer — 1 second at 100 MHz

Write LOAD and RELOAD byte-by-byte, MSB first. 100,000,000 = 0x05F5E100: byte 0 = 0x05, byte 1 = 0xF5, byte 2 = 0xE1, byte 3 = 0x00. Also configure mtvec, mie bit 11, and mstatus bit 3 (see Section 4.1) before enabling the timer.

```
li    t0, 0xe0002000

# Enable interrupt routing
li    t1, 1
sw    t1, 0x40(t0)      # EV_ENABLE

# Write LOAD: 100,000,000 = 0x05F5E100 (MSB first)
li    t1, 0x05
sw    t1, 0x00(t0)      # LOAD0: bits[31:24]
li    t1, 0xF5
sw    t1, 0x04(t0)      # LOAD1: bits[23:16]
li    t1, 0xE1
sw    t1, 0x08(t0)      # LOAD2: bits[15:8]
sw    x0, 0x0C(t0)      # LOAD3: bits[7:0] = 0

# Write RELOAD (same values – repeats every second)
li    t1, 0x05
```

```

sw    t1, 0x10(t0)      # RELOAD0
li    t1, 0xF5
sw    t1, 0x14(t0)      # RELOAD1
li    t1, 0xE1
sw    t1, 0x18(t0)      # RELOAD2
sw    x0, 0x1C(t0)      # RELOAD3 = 0

# Start the timer
li    t1, 1
sw    t1, 0x20(t0)      # EN

```

6.2 ISR Pattern — Servicing and Clearing the Timer Interrupt

Inside your trap handler check `mcause` for `0x8000000B` (bit 31 set = interrupt, lower 11 = CPU line 11). After servicing, write 1 to `EV_PENDING` (offset `0x3C`) to clear the flag. The write must happen before `mret` or the CPU immediately re-enters the ISR.

```

machine_trap_handler:
    addi sp, sp, -64
    sw    ra, 0(sp)
    sw    t0, 4(sp)      # ... save t1-t6, a0-a7 similarly ...

    csrr  t0, mcause
    li    t1, 0x8000000B  # bit31=interrupt, source=11 (LiteX Timer)
    bne   t0, t1, .Ldone

    # --- service the timer interrupt ---
    # ... your code here ...

    # Clear EV_PENDING ← REQUIRED before mret
    li    t0, 0xe000203C  # TIMER_BASE + 0x3C
    li    t1, 1
    sw    t1, 0(t0)

.Ldone:
    lw    ra, 0(sp)
    lw    t0, 4(sp)
    addi  sp, sp, 64
    mret

```

NOTE Using `cs281.inc`: the equates `TIMER_BASE`, `TIMER_LOAD0–LOAD3`, `TIMER_RELOAD0–RELOAD3`, `TIMER_EN`, `TIMER_EV_PENDING`, and `TIMER_EV_ENABLE` are all defined in `hardware/lib/cs281.inc`. Pre-computed byte slices of `TIMER_FREQ` (`TIMER_FREQ_B0–B3`) are also available.

8. GPIO Output — LEDs

The GPIO Output peripheral drives four on-board LEDs. A single 32-bit register controls all four outputs simultaneously using a bitmask. Bits 31:4 are ignored.

Base Address: 0xe0015000

8.1 DATA Register (offset 0x00)

Bits	Name	Description
31:4	—	Reserved. Always write 0.
3	LED3	1 = LED3 ON, 0 = LED3 OFF
2	LED2	1 = LED2 ON, 0 = LED2 OFF
1	LED1	1 = LED1 ON, 0 = LED1 OFF
0	LED0	1 = LED0 ON, 0 = LED0 OFF

8.2 Usage Examples

```

li    t0, 0xe0015000
li    t1, 0x1          # turn ON LED0 only
sw    t1, 0(t0)

li    t1, 0x5          # turn ON LED0 + LED2 (bits 0 and 2)
sw    t1, 0(t0)

sw    zero, 0(t0)      # turn OFF all LEDs

```

NOTE Writing a new value replaces the entire register. To set a single LED without disturbing others, read-modify-write: `lw t2,0(t0) / or t2,t2,t1 / sw t2,0(t0)`.

9. GPIO Input — Buttons and DIP Switches

The GPIO Input peripheral reads four digital inputs: two push buttons and two DIP switch positions. Any state change on any input triggers an interrupt on CPU line 12.

Base Address: 0xe0015400 | IRQ: CPU line 12 (state change)

9.1 DATA Register (offset 0x00) — Read Only

Bits	Name	Active	Description
31:4	—	—	Reserved. Reads as 0.
3	DIP_SW1	1	DIP switch 1 position. 1 = switch ON, 0 = switch OFF.
2	DIP_SW0	1	DIP switch 0 position. 1 = switch ON, 0 = switch OFF.
1	BTN1	1	Push button 1. 1 = pressed, 0 = released.
0	BTN0	1	Push button 0. 1 = pressed, 0 = released.

9.2 Polling Example — Wait for BTN0

```
li    t0, 0xe0015400
.wait_btn:
lw    t1, 0(t0)          # read GPIO_IN
andi  t1, t1, 0x1        # mask BTN0 (bit 0)
beqz  t1, .wait_btn      # loop until pressed
```

9.3 Driving Inputs from the Renode Monitor

GPIO inputs have no physical hardware; they are driven from the Renode monitor console. Connect to the monitor terminal (the terminal where Renode runs) and type:

```
gpio_in SetGPIO 0 true    # press BTN0
gpio_in SetGPIO 0 false   # release BTN0
gpio_in SetGPIO 2 true    # turn DIP_SW0 ON
```

NOTE Pin numbers: 0 = BTN0, 1 = BTN1, 2 = DIP_SW0, 3 = DIP_SW1.

10. 7-Segment Display

The 7-segment display is driven by an 8-bit GPIO output register. Each bit independently controls one segment of the display. Digits are formed by writing the correct bitmask for the desired segments.

Base Address: 0xe0004400

10.1 Segment Bitmask (DATA register, offset 0x00)

Bit	Name	Physical Segment	Description
7	SEG_DP	Decimal point	1 = dot lit
6	SEG_G	Middle bar	1 = lit
5	SEG_F	Top-left bar	1 = lit
4	SEG_E	Bottom-left bar	1 = lit
3	SEG_D	Bottom bar	1 = lit
2	SEG_C	Bottom-right bar	1 = lit
1	SEG_B	Top-right bar	1 = lit
0	SEG_A	Top bar	1 = lit

10.2 Digit Patterns

Digit	Hex Value	Segments Active
0	0x3F	A B C D E F
1	0x06	B C
2	0x5B	A B D E G
3	0x4F	A B C D G
4	0x66	B C F G
5	0x6D	A C D F G
6	0x7D	A C D E F G
7	0x07	A B C
8	0x7F	A B C D E F G
9	0x6F	A B C D F G

10.3 Example — Display the digit "3"

```
li    t0, 0xe0004400
li    t1, 0x4F          # digit 3: A+B+C+D+G
sw    t1, 0(t0)
```

NOTE The constants `SEG7_DIGIT_0` through `SEG7_DIGIT_9` are defined in `hardware/lib/cs281.inc` for use in assembly programs.

11. Buzzer

The buzzer is a single-bit GPIO output. Writing 1 activates the buzzer; writing 0 deactivates it. To produce a tone, toggle the output rapidly in a loop or timer ISR.

Base Address: 0xe0004000

Bits	Name	Description
31:1	—	Reserved. Always write 0.
0	BUZZER	1 = buzzer ON, 0 = buzzer OFF

11.1 Example — 500 Hz Tone (busy-loop, approximate)

```
li    t0, 0xe0004000
li    t2, 1000          # toggle count
.tone_loop:
li    t1, 1
```

```
sw    t1, 0(t0)      # buzzer ON
li    a0, 5000        # half-period delay
call  delay
sw    zero, 0(t0)     # buzzer OFF
li    a0, 5000
call  delay
addi  t2, t2, -1
bnez  t2, .tone_loop
```

12. Register Quick Reference

All addresses shown are absolute byte addresses. All registers are 32-bit; use lw / sw unless noted.

Address	Name	R/W	Description
0xe0001800	UART_RXTX	R/W	UART data — write byte to TX, read byte from RX
0xe0001804	UART_TXFULL	R	1 = TX FIFO full, do not write
0xe0001808	UART_RXEMPTY	R	1 = RX FIFO empty, no data available
0xe0001810	UART_EV_PENDING	R/W	UART interrupt pending — write 1 to clear
0xe0001814	UART_EV_ENABLE	R/W	UART interrupt enable
0xe0002000	TIMER_LOAD	R/W	Load value MSB byte (LOAD0). Write LOAD0-3 MSB-first. See Section 6.
0xe0002004	TIMER_RELOAD	R/W	Reload value MSB byte (RELOAD0). Write RELOAD0-3 MSB-first. See Section 6.
0xe0002020	TIMER_EN	R/W	Timer enable (1=run)
0xe0002024	TIMER_UPDATE	W	Write 1 to latch current count into VALUE sub-registers.
0xe0002028	TIMER_VALUE	R	Current count (latch first)
0xe000203C	TIMER_EV_PENDING	R/W	Timer interrupt pending — write 1 to clear
0xe0002040	TIMER_EV_ENABLE	R/W	Timer interrupt enable
0xe0004000	BUZZER	W	Bit 0: 1=ON, 0=OFF
0xe0004400	SEG7	W	Bits 0-7: segments A-G, DP
0xe0005000	CLINT_MSIP	R/W	Software interrupt pending (bit 0)
0xe0009000	CLINT_MTIMECMP	R/W	Timer compare low word
0xe000BFF8	CLINT_MTIME	R	Current time low word
0xe0015000	GPIO_OUT	R/W	LED outputs — bits 0-3 = LED0-LED3
0xe0015400	GPIO_IN	R	Board inputs — bits 0-3 = BTN0,BTN1,DIP_SW0,DIP_SW1